

COSE451 Software Security - HW 01: Code Review and Unit Tests

Release date: 28th September 2026 (Monday)

Submission deadline: 18th October 2026 (Sunday), 23:59:59 KST

This lab consists of two parts:

- Review source code and find bugs
- Familiarize with unit tests both for functionality and edge cases

We will cover the unit-test part in class shortly, so focus on the bug-finding part first.

Grading

The lab is graded over 30 points with the subsequent division, each explained in detail in its corresponding section.

- 20 points for bug finding (11 categories x 2 points, capped at 20)
- 10 points for unit tests

Your submission is additionally evaluated with hidden tests that are not part of this handout, so make sure your fixes and your tests work in general, not only on the inputs given here.

Make sure **all** source files you submit are formatted using `clang-format` and LLVM style (`--style=LLVM`) before submission. `clang-format` can be obtained on Ubuntu by installing the package `clang-format`.

The code that isn't linted/formatted properly or isn't compilable will subtract from 5 points up to all the grade

Submission Instructions

Please submit a zip file `<KU_id>.zip` within the deadline (generate it by running `zip -r <KU_id>.zip reports src`), an example valid `<KU_id>` is `2024000000`.

The archive holds `src/` (the code you work on), `reports/` (with the worked example), this document as `HW1.pdf`, `SETTINGS.md`, and the three Docker files. Keep them together: the Docker setup expects to be started from the directory that holds `src/`.

1. Extract the archive
2. Apply your fixes directly in `src/`.
3. Put your reports in the `reports/` directory that came with the archive.

4. For every discovered bug, save a file, named `bug_X.md`, where X is a bug identifier of your choosing. Start at 1: `bug_0.md` is the worked example we shipped, leave it as it is.
5. Create a new zip archive with the two top-level directories, `src/` and `reports/` and upload it to the assigned submission form on LMS.

If a bug report needs a proof-of-concept image, put it in `src/test_imgs/`, which keeps all images together. Any other location inside the archive is fine as well, as long as every `bug_X.md` states the path of the file it refers to. Reports and source comments are expected in English whenever you can manage it.

Please follow the following checklist before submission. Do not leave it till the last minute.

- The folder structure should be EXACTLY as provided (see below).
- No source file names should be modified
- Remove all compiled/generated .o,.a files and executables (`make clean` should do this job for you)
- Remove all unnecessary files (ex. `source_file.bak.v1`)

```
<KU_id>.zip
├─ reports
│   └─ bug_*.md
└─ src
    ├─ *.c|h
    └─ test_imgs
        ├─ *.png
        └─ (your proof-of-concept images, if any)
```

If you have any doubts, don't hesitate to contact with us earlier rather than later.

Y0L0 PNG library

Utility Programs Description

Several programs have been implemented to showcase the capabilities of the Y0L0 PNG format:

- `checkerboard`
- `circle`
- `crop`
- `filter`
- `mosaic`
- `rect`
- `resize`
- `solid`

Solid

Usage: `solid output_file height width hex_color`

Output: A PNG file consisting of a solid block of color. If `output_file` does not already end in `.png`, the extension is appended, so `solid poc 8 8 ff8000` writes `poc.png`.

Checkerboard

Usage: `checkerboard output_file height width square_width hex_color1 hex_color2`

Output: A PNG file consisting of squares of alternating colors.

Filter

Usage: `filter input_file output_file filter_name [filter_arg]`

Output: Processes the input image using the specified filter.

Grayscale

Usage: `filter input_file output_file grayscale`

Output: The grayscale version of the input image.

Negative

Usage: `filter input_file output_file negative`

Output: The negative version of the input image.

Blur

Usage: `filter input_file output_file blur blur_radius`

Output: The blurred version of the image where every pixel is replaced by an average value of its neighborhood.

Alpha

Usage: `filter input_file output_file alpha hex_alpha`

Output: Sets the alpha channel of every pixel to the value of `hex_alpha`.

Sharpen

Usage: `filter input_file output_file sharpen`

Output: The sharpened version of the input image. This filter is not implemented yet; implementing it is part of this assignment (see [Additional filters](#)).

Circle

Usage: `circle input_file output_file radius center_x center_y hex_color`

Output: Draws a specified circular line over the input image.

Rect

Usage: `rect input_file output_file top_left_x top_left_y bottom_right_x bottom_right_y hex_color`

Output: Draws the specified rectangle over an input image.

Resize Image

Usage: `resize input_file output_file float_factor`

Output: Resizes the input by the specified factor.

Crop

Usage: `crop input_file output_file x y width height`

Output: A PNG file holding the `width x height` region of the input image whose top-left corner is the pixel `(x, y)`. The requested region has to lie inside the input image. Every crop is appended as one line to the file `crop_history.log` in the current directory.

Mosaic

Usage: `mosaic input_file output_file block_size`

Output: A pixelated version of the input image. The image is partitioned into `block_size x block_size` blocks, and every pixel of a block is replaced by the average color of that block. Blocks at the right and bottom border are cut off by the image edge, and then only the pixels inside the image count towards the average.

Building

Environment Setup Options

This assignment can be completed using two methods. For detailed setup instructions, please refer to the `SETTINGS.md` file.

1. Docker Environment (Recommended)

- Ensures consistent development environment
- Automatic dependency resolution

2. Local Environment

- Direct system access
- Faster compilation speed

Build Instructions

Running `make` builds the PNG library, the eight utility programs and the unit tests. Building depends on `zlib` and on the Check unit-testing framework; on Ubuntu they are the packages `zlib1g-dev`, `check` and `build-essential` (`pkg-config` is used to locate Check). All of them are already installed in the provided Docker image.

- `make` or `make all` - build everything (`./solid`, `./filter`, ..., `./tests`)
- `make utils` - build only the utility programs
- `make <program_name>` - build a single program
- `make asan` - build every program and the tests with AddressSanitizer and UndefinedBehaviorSanitizer, as `./solid_asan`, `./filter_asan`, ..., `./tests_asan`. These binaries report memory errors and undefined behaviour that a normal build does not report. Use them to confirm a bug you suspect, and to confirm your fix. They have their own names, so they never overwrite the normal binaries.
- `make clean` - remove everything that was built

Y0L0 PNG Format

Y0L0 PNG format is a subset of the PNG file format. It consists of a PNG file signature, followed by mandatory PNG chunks:

- IHDR chunk
- Optional PLTE chunk
- One or more IDAT chunks
- IEND chunk

All multibyte data chunk fields are stored in the big-endian order (e.g. 4-byte integers, CRC checksums). IHDR must follow the file signature. If the palette is used to denote color, PLTE chunk must appear before IDAT chunks. IDAT chunks must appear in an uninterrupted sequence. The image data is the concatenation of the data stored in all IDAT chunks. IEND must be the last chunk in an image. All other chunk types are simply ignored.

PNG files from the web or generated by *real* programs (e.g., screenshot) will usually not work with this library. You should be able to generate example PNG files with the `solid` or `checkerboard` program.

Y0L0 PNG File Signature

Y0L0 PNG files start with the byte sequence: `137 80 78 71 13 10 26 10`

Y0L0 PNG Chunks

Y0L0 PNG chunks have the following structure:

- Length (4 bytes) denotes the length of the data stored in the chunk

- Chunk type (4 bytes) identifies the chunk type (IHDR, PLTE, IDAT or IEND). The type is encoded as a 4 byte sequence.
- Chunk data (Length bytes) stores the actual chunk data
- CRC code (4 bytes) is a checkcode that is calculated over chunk type and chunk data

All fields are consecutive and in the given order.

IHDR Chunk

IHDR chunk must appear as the first chunk following the file signature. It has the type `IHDR` and the following structure of the chunk data:

- Width (4 bytes)
- Height (4 bytes)
- Bit depth (1 byte)
- Color type (1 byte)
- Compression method (1 byte)
- Reserved (2 bytes)

All fields are consecutive and in the given order.

Bit Depth

The only supported bit-depth is 8 bits (1 byte). This refers to 1 byte per color channel in the RGBA color mode, and to 1 byte per palette index in PLTE color mode.

Color Type

Color type field denotes the way color data is stored in the image. The only supported values are 3 (palette) and 6 (RGBA).

- Palette denotes that we expect to find a PLTE chunk in the image. In the IDAT chunk data colors are not represented as RGBA tuples, but as indices in the palette table. Every offset has the length of bit-depth. If the pixel has the color of 0x123456, and the palette has the color {R:0x12, G:0x34, B: 0x56} at the position 5, the value 5 will be stored in the image.
- RGBA mode represents colors in IDAT data as a sequence of 4 values per pixel, each one bit-depth in size. Every value corresponds to the intensity of a RGBA (red-green-blue-alpha) channel.

Compression Method

The only supported compression method is the deflate algorithm signified by value 0.

Reserved

Reserved for future use.

PLTE Chunk

PLTE chunk must appear before the IDAT chunk if the palette is used to encode color. Its type is encoded with `PLTE`. The chunk data is an array of PLTE entries which are defined as:

- Red (1 byte)
- Green (1 byte)
- Blue (1 byte)

The length field of the PLTE chunk needs to be divisible by 3.

IDAT Chunk

The type of the IDAT chunk is encoded by the bytes `IDAT`. If multiple IDAT chunks exist, they must all occur in sequence. The image data is the concatenation of the data stored in all the IDAT chunks in the order in which they appeared. It is compressed using the deflate algorithm and is stored in the zlib file format:

- Compression type and flags (1 byte)
- Flags and check bits (1 byte)
- Compressed data (n bytes)
- Adler-32 checksum (4 bytes)

This data is used as an input for the inflate algorithm and is handled by zlib. Inflated data is stored left-to-right, top-to-bottom. Every row (scanline) begins with a byte denoting the start of the line. This byte must be 0. Pixel information for the scanline follows this byte. It consists either of the palette index (1 byte), or of the RGBA pixel value (4 bytes), depending on the color format.

All fields of structures are consecutive and in the given order.

IEND Chunk

IEND chunk appears as the last chunk in the file. Its type is `IEND`. It stores no data and the length field is 0.

PNG Parser Interface

PNG Parser interface consists of functions:

- `int load_png(const char *filename, struct image **img)` loads a png file denoted by the filename and writes a pointer to struct image into the memory pointed to by img. Please remember to free both img and img->px after you are finished using the image. Also remember to do it in the correct order. This function returns 0 on success and a non-zero value on failure.
- `int store_png(const char *filename, struct image *img, struct pixel *palette, uint8_t palette_length)` which stores an image into the file. Note the type of `palette_length`: a palette can hold at most 255 entries. If the palette argument is NULL, the image is stored in the RGBA format. Otherwise, the palette is used to index colors.

- `struct image` stores the height and the width of an image, as well as a flattened array of pixels.

```
struct image {
    uint16_t size_x;
    uint16_t size_y;
    struct pixel *px;
};
```

- `struct pixel` represents a pixel as a RGBA tuple.

```
struct pixel {
    uint8_t red;
    uint8_t green;
    uint8_t blue;
    uint8_t alpha;
};
```

Code Review

Bugs and Fixes

In this lab, your task is to review the code in order to find bugs, trigger them, and write the fixes for these bugs. There are at least 20 bugs but less than 25.

One bug means one root cause: if the same mistake shows up in several lines of the same function, report it once and fix it everywhere. Conversely, two different mistakes in the same line are two bugs.

We don't look only for bugs that you can clearly end up in a crash (e.g. `SIGSEGV`), but which diverts from the program's design. Consider that there may be bugs that don't seem exploitable at first impression, but that without much difficulty can suppose a threat.

Finding Bugs

1. Go through the code and identify locations where faulty behavior can occur.
2. For each identified location, create a test case that triggers the bug.
3. File a bug report for each bug, following the sample file `bug_0.md`. Keep one bug per file, and state the file and the line numbers of the faulty code, the category, the command that triggers it and the fix you applied.

Only bugs in the following files will count towards the grade in this lab:

- `checkerboard.c`
- `circle.c`
- `crop.c`
- `filter.c`

- `mosaic.c`
- `rect.c`
- `resize.c`
- `solid.c`

However, any bugs found in the PNG parsing library (`pngparser.c`) do **NOT** count but may come in useful later in the course. Example bugs will not be graded, but students are encouraged to fix them on their own.

Fixing Bugs

For each discovered bug, identify the culprit (e.g., uninitialized variable), and replace it with code that produces results which match expected behavior. Expected behavior can be inferred from the description of different functions and tools. Exercise common sense and write the simplest fix you can think of for each bug.

Change the code itself. A comment that describes the fix, or one that explains what should happen while the faulty line stays in place, earns no points. We rebuild the `src/` you submit and run it, so it has to compile and behave correctly. The example bug marked in `filter.c` shows the reporting format. It is the one bug we do not grade, and its code is left faulty on purpose.

Your fixes will be verified for functionality. A fix that removes the bug but breaks documented behaviour, for example by rejecting input the program should accept, does not count.

Grading

We will evaluate for each bug the effectiveness of the patch that you submit and a bug report. We classify the bugs in 11 categories and each category is worth 2 points, so by solving fully 10 categories should be enough to obtain the maximum grade; but that extra category albeit won't count directly, will be taken in consideration. There are categories with only a bug, but there are also categories with multiple bugs, in the case you don't find all bugs in a category, your grade for that category will be $\text{bugs_found} / (\text{bugs_found} + 1)$; if all bugs are discovered, it will give out the whole score for it. The categories are the next ones:

- Unchecked system call returning code
- Stack buffer overflow/underflow
- Command injection
- Arithmetic overflow/underflow
- Heap overflow/underflow
- Temporal safety violation
- Local persisting pointers
- String vulnerability
- Iteration errors
- Wrong operators
- Type error

Any bug outside of the previous categories may count if the teaching staff considers it so.

Unit Tests

Unit testing is a form of testing that focuses on testing 'small' regions of code extensively. This is often part of a code development strategy known as Test-driven development.

As opposed to end-to-end testing (which treats the program as a black-box and looks at the final outputs for test inputs) unit testing generally follows a white-box approach. Thus, unit tests are able to run small snippets of code, and deeply inspect program state. For example, unit tests are able to read and check state within data structures and objects. When testing a single function, unit-testing also allows testing of obscure paths which might be harder to trigger in an end-to-end test. However, unit-testing is not an exhaustive approach and is only as complete as the number of edge-cases that the programmer can envision and implement tests for.

The exact use of unit-tests depends on the requirements of the program being tested, but typically includes tests for functionality on known, simple inputs and outputs, and tests for corner cases and errors. The functionality tests also allow the programmer to check when program behavior deviates from expected, and thus, to debug their programs. This is where unit testing fits into the development <=> testing cycle. The tests for corner cases and errors are used to check for bugs on the error handling paths.

You have to implement unit tests for the existing filters (consider that the given implementation may be faulty). You are also required to implement one additional filter, sharpen, for which we provide part of the tests and the reference images.

Check: The unit testing framework

In this lab, we shall use the Check unit testing framework (<https://libcheck.github.io/check/>) for C. The framework allows the user to define tests within `START_TEST` and `END_TEST` macros. Within each test, checks are implemented by `ck_assert_*()` function calls. An example test for a function implementing addition of two integers might look like this:

```
START_TEST(addition_test) {
    /* Testing equality between two integers */
    ck_assert_int_eq(sum(1, 2), 1 + 2);

    /* Test inequality between two integers */
    ck_assert_int_gt(sum(1, 2), 2);
}
END_TEST
```

An example output for a unit test run is shown below. Note that of four tests, only one passed. The failed tests have failed for 3 different reasons. `failed_test` failed an equality check on line 11 (depends on program logic). `crashing_test` generated a SEGFAULT. `timeout_test` timed-out after the default timeout of 4 seconds (see `CK_DEFAULT_TIMEOUT`).

```
Running suite(s): Example testsuite
25%: Checks: 4, Failures: 1, Errors: 2
tests.c:6:P:Example tests:successful_test:0: Passed
tests.c:11:F:Example tests:failed_test:0: Assertion 'fancy_sum(1u, 2u) == 1u + 2u'
failed: fancy_sum(1u, 2u) == 3221342974, 1u + 2u == 3
tests.c:15:E:Example tests:crashing_test:0: (after this point) Received signal 11
(Segmentation fault)
tests.c:20:E:Example tests:timeout_test:0: (after this point) Test timeout expired
```

You will find more example tests implemented in the file `tests.c`. Your work is to fill in the bodies marked `TODO`; the eight tests listed below are the graded ones, and you do not have to invent additional tests.

You can install `check` using the command `apt install check` on Ubuntu.

Writing unit tests

The file `tests.c` contains twelve tests. Four of them are already implemented and serve as examples (`gray_weight_extremes`, `gray_zero_weights`, `sharpen_tiny_image` and `sharpen_checkerboard`). The other eight are marked `TODO` and are your work; each one is worth 1 point. All twelve are already registered in `main()`, so you only fill in the bodies (plus the radius list in `main()`, which belongs to `blur_radius_limits`).

The comment above each test states exactly what it has to check. In short:

Test	What it tests
<code>gray_reference_images</code>	grayscale of <code>desert.png</code> and <code>summer.png</code> matches the shipped reference images, alpha untouched (loop test, <code>_i</code> selects the image pair)
<code>invert_roundtrip</code>	negating a black image of random size gives white, negating again gives black, alpha untouched
<code>invert_empty_image</code>	the negative filter does not crash on a 0x0 image
<code>blur_small_kernel</code>	blur of a 3x3 image with a single white (180) pixel, for the radii 0, 1, 2 and 3
<code>blur_radius_limits</code>	blur does not crash for 20 extreme radii, listed in <code>main()</code> (loop test)
<code>alpha_overwrite</code>	the transparency filter sets the alpha channel of every pixel and changes nothing else
<code>alpha_null_argument</code>	the transparency filter dies with <code>SIGSEGV</code> when the transparency argument is <code>NULL</code> (pass an image with at least one pixel; the filters are not required to tolerate <code>NULL</code> , so do not add a defensive check while fixing bugs)
<code>sharpen_reference_images</code>	your <code>filter_sharpen</code> reproduces the three shipped reference images (loop test)

Three things about the starting state, so that you can tell our bugs from your own mistakes:

- On a fresh checkout `./tests` fails almost everywhere, including the two implemented grayscale examples. Built with the container's gcc, the grayscale filter crashes on one of the bugs you are asked to find; on other compilers the same bug may not crash at all, which is a reason to work in the provided image (see `SETTINGS.md`). Both examples pass once that bug is fixed.
- `alpha_overwrite` fails until you fix the iteration bug in the transparency filter, and the three sharpen tests stay red until you implement `filter_sharpen`.
- The two blur tests pass only once the blur filter reads inside the image. The cause is the example bug marked in `filter.c`. We do not grade it as a bug report, but the tests need it fixed.

Notes that apply to several of the tests:

- Do not forget to seed the random number generator before generating random values.
- A filter is allowed to replace `img.px` with a buffer of its own, so pass pixel buffers that live on the heap, and re-read `img.px` after the call instead of keeping the pointer you passed in.
- Use `duplicate_img` when a test needs the original pixels for comparison, or when a loop test must not disturb the image used by the next iteration.
- For the loop tests there is an implicitly defined variable `_i`, the current iteration. Look at `sharpen_checkerboard` and at the `tcase_add_loop_test` calls in `main()` for the syntax.
- `blur_radius_limits` runs over the 20 radii listed in the `TODO` comment in `main()`; fill the array with exactly those values. Each index is a separate test case, so the order does not matter. We know that not all of them make sense for a blur. Use them nonetheless, but take care not to write down an expression that overflows a signed integer (`INT_MAX + 1` and `INT_MIN - 1` are undefined behaviour, which is why they are not in the list).
- `alpha_null_argument` is registered with `tcase_add_test_raise_signal()`, so Check treats the segmentation fault as the expected outcome. Run it with the plain `./tests` binary: in `./tests_asan` the sanitizer intercepts the fault and exits before the signal reaches Check, which makes the test fail.

Additional filters

You also have to implement one additional filter: **sharpen**, which increases the local contrast of the image, so that edges look crisper. The stub is in `filter.c`:

```
void filter_sharpen(struct image *img, void *unused);
```

The `unused` argument exists so that the filter has the same signature as the others; ignore it. A correct implementation is worth 2 points, and the unit tests `sharpen_reference_images`, `sharpen_checkerboard` and `sharpen_tiny_image` check it against the shipped reference images.

Sharpen filter

Sharpening is a convolution with the 3x3 kernel

```
K =
  0  -1  0
 -1  5  -1
  0  -1  0
```

For every pixel `(y, x)` and for each of the red, green and blue channels, the new value is

```
acc = sum over dy, dx in {-1, 0, 1} of K[dy + 1][dx + 1] * src[y + dy][x + dx]
```

where `src` is the **unmodified** source image, and the result is clamped into `[0, 255]` :

```
new_value = acc < 0 ? 0 : (acc > 255 ? 255 : acc)
```

The details, all of which the reference images depend on:

- **Read from a copy.** Every neighbour has to be read from the source image as it was before the filter started. If you write your results back into the image while you compute them, later pixels read values that were already sharpened, and the output will not match the reference.
- **Bound the indices.** For pixels on the border the formula asks for pixels outside the image. Bound row indices into `[0, size_y)` and column indices into `[0, size_x)`, that is, repeat the border pixel: `src[-1][-1]` becomes `src[0][0]`, `src[3][-1]` becomes `src[3][0]`, and `src[-1][4]` becomes `src[0][4]`.
- **The alpha channel is copied unchanged.** It is never sharpened.
- **An image with `size_x == 0` or `size_y == 0`** has to be left alone; return immediately, without allocating or dereferencing anything.
- **After the call, `img->px` has to be a valid heap buffer holding the result.** Whether you write the result back into the original buffer or install a new one (freeing the old one, as `filter_blur` does) is up to you.

The kernel weights add up to 1, so a uniform image comes out unchanged. `sharpen_tiny_image` checks that on a 1x1 image; an unimplemented filter also passes it.

The reference images `desert_sharpen.png`, `summer_sharpen.png`, `ck_sharpen.png` and `sharpen_small_out.png` in `test_imgs/` were produced with the reference implementation. `sharpen_small_in.png` is a 5x4 image, small enough to work out by hand when you debug your filter.

Common gotchas

The behavior of `check` depends on an environment variable `CK_FORK`. If this variable is `yes` or undefined, Check forks before running each tests, thereby running each test in a different process. This allows each test to be independent, and bugs in one test do not affect other tests. This means that when trying to debug tests with `gdb`, you will see the below output. Despite setting a breakpoint, you might see that the breakpoint is never hit. This is because `gdb` continues debugging the parent process by default on a fork, while the actual tests run in children.

Warning: `alpha_null_argument` is supposed to die with `SIGSEGV`. With `CK_FORK=no` that signal kills the whole test binary (exit status 139) and no results are printed, so use `CK_FORK=no` only while debugging one of the other tests.

```
(gdb) b tests.c:305
Breakpoint 1 at 0x6ee9: file tests.c, line 305.
(gdb) r
Starting program: /homework/src/tests
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Running suite(s): lib-Y0l0 tests
[Detaching after fork from child process 6645]
[Detaching after fork from child process 6646]
[Detaching after fork from child process 6647]
[Detaching after fork from child process 6648]
[Detaching after fork from child process 6649]
...
```

To debug tests with `gdb`, either set the environment variable `CK_FORK` to `no`. You can do this from within `gdb` by running the command `set env CK_FORK=no`. This has the advantage that all your tests run in one process and your breakpoints will be hit. However, this can lead to bugs from one test affecting other tests. Specially for bugs involving the memory allocator (`malloc` / `free` et al.) these bugs can get really difficult to find/debug.

The other option is to ask `gdb` to follow the child on a fork. This can be done with the command `set follow-fork-mode child`. This must be done just before the test which you want to debug.

Example script:

```

file tests

b fork
ignore 1 2
commands
set follow-fork-mode child
info proc
catch syscall clone
end

run

commands 2
info proc
b gray_weight_extremes_fn
b gray_zero_weights_fn
b invert_empty_image_fn
b sharpen_tiny_image_fn
b alpha_null_argument_fn
b gray_reference_images_fn
b invert_roundtrip_fn
b blur_small_kernel_fn
b blur_radius_limits_fn
b alpha_overwrite_fn
b sharpen_reference_images_fn
b sharpen_checkerboard_fn
c
end

c

```

In this script, we set a breakpoint on `libc's fork` function, and `ignore` the first 2 times we encounter it. Tailor this number to get to the correct test. This breakpoint also runs some commands when hit, which we will discuss later.

Note: At this point, the `fork` breakpoint has not been hit, and its commands have not been executed.

Then, we issue the `run` command, which starts executing the `tests` executable.

When it hits the breakpoint, `gdb` runs the commands set on `fork` breakpoint. Let us take a better look at what these commands were:

- it changes `gdb's` forking mode to debug the child
- `info proc` shows that we are still running in the parent.
- sets a catchpoint, which is used to stop just before the `clone` syscall.

At this point, we have a catchpoint (also breakpoint 2) and we can set commands for it. The `commands 2` block shows these. We'll ignore them for a bit.

The final `c` command tells `gdb` to resume execution. This will continue until it hits the catchpoint, and executes the commands we set up earlier:

- sets up breakpoints for all tests (you can shorten this list, only breaking on the particular test you are interested in). Note the `_fn` suffix: the `START_TEST(name)` macro defines the test function as `name_fn`, while `name` itself is a variable, so `b name` does not stop in the test body.
- `c` command, which resumes execution after the catchpoint

`gdb` continues after the catchpoint. The `syscall` causes a child process to be spawned, which `gdb` follows. The child eventually executes a test, and the corresponding breakpoint is hit.

Grading

The correct implementation of the sharpen filter is worth 2 points, and each of the eight unit tests marked `TODD` is worth 1 point, for 10 points in total. A test counts only if it checks what its comment asks for. An empty body, or one that passes whatever the filter does, earns nothing.